

IntentKeeper: Validating Developer Intentions in Solidity via Intent Annotations to Mitigate Numeric Logic error

Inseong Jeon¹, Sundeuk Kim¹, Hyunwoo Kim¹, Hoh Peter In^{1*}

^{1*}Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Logic errors in smart contracts—where program behavior diverges from developer intent—are among the most challenging vulnerabilities to detect. Unlike well-defined patterns such as reentrancy or integer overflow, logic errors require knowledge of developer intent, which is absent from the code itself. Existing approaches, including static analyzers, dynamic testing, and LLM-based methods, cannot reliably identify such errors without explicit intent representation. This paper presents INTENTKEEPER, a tool that enables developers to specify and validate their intentions for numeric operations during smart contract development. INTENTKEEPER introduces an intent annotation model with two annotation types: **@During** annotations specify intended relationships at specific program points within a function, while **@Post** annotations specify intended relationships after function execution. The underlying analysis uses interval-domain abstract interpretation to compute value ranges at each program point and check whether they satisfy the specified intentions. We evaluated INTENTKEEPER on real-world smart contracts, demonstrating its effectiveness in validating developer intentions and providing immediate feedback during development.

Keywords: Smart Contract Development, Solidity, Numeric Logic Error, Intent Annotations, Abstract Interpretation, Developer Tools

1 Introduction

Smart contracts are immutable once deployed, making security assurance during development critical. Among vulnerability types, logic errors present a distinct challenge: they arise when program behavior diverges from developer intent, a discrepancy that is inherently invisible to compilers and external reviewers alike (Soud et al., 2024).

A recent empirical study (Chaliasos et al., 2024) confirms this challenge in practice. Practitioners identified logic errors as the most difficult vulnerability type to detect, reporting that existing security tools are least effective against them. This gap exists because current approaches—static analyzers, dynamic testing, and even recent LLM-based methods—predominantly target well-defined vulnerability patterns. Logic errors, however, require knowledge of developer intent, which is absent from the code itself. Various research efforts have attempted to detect specific logic errors, but without explicit intent representation, they must rely on predefined detection patterns. Such patterns can identify violations of specific rules, but not actual deviations from intended behavior. Consequently, even when these tools report no issues, developers cannot be certain that their code behaves as intended.

The same study (Chaliasos et al., 2024) also provides insight into developer needs. Developers expressed a preference for development-integrated tools such as IDE extensions and linters over standalone analyzers. Moreover, when asked about the most valued properties of security tools, developers prioritized low false negatives (92.3%) over low false positives (50%), indicating a strong preference for tools that do not miss real issues, even at the cost of occasional false alarms. This suggests a need for development-phase tools that can provide sound guarantees about code behavior. To determine where such intent specification is most needed, we analyzed 1,023 non-trivial functions from DAppSCAN-source (Zheng et al., 2024) (see Table ??). Numeric variable types appear in 82% of these functions, with `uint` alone used in 78%. Since numeric computations form the core of smart contract business logic—handling token transfers, fee calculations, and balance updates—explicitly specifying developer intent for these operations becomes essential.

Based on these observations, this paper presents INTENTKEEPER, a tool that enables developers to specify and validate their intentions for numeric operations during the development phase. INTENTKEEPER introduces an intent annotation model consisting of two annotation types: `@During` annotations specify intended relationships that should hold at specific program points within a function, while `@Post` annotations specify intended relationships that should hold after the function completes execution. Developers write these annotations alongside their code, and INTENTKEEPER validates whether the program’s abstract execution satisfies these specified intentions.

The underlying value analysis is performed using interval-domain abstract interpretation, building upon our previous work on SOLQDEBUG (Jeon et al., 2025), an interactive debugger for Solidity. Through `@Debugging` annotations, developers specify value ranges for variables, and the analyzer computes abstract values at each program point. INTENTKEEPER extends this capability by checking whether these computed values satisfy the developer-specified intent annotations, providing immediate feedback during development rather than requiring completed code.

This paper makes the following contributions:

- We propose a classification of numeric logic errors based on function execution semantics, categorizing known vulnerabilities by their occurrence points within single-transaction execution.
- We present an intent annotation model that enables developers to express intended numeric relationships at both statement-level (`@During`) and function-level (`@Post`) granularity.
- We develop validation algorithms for each annotation rule that leverage interval-domain abstract interpretation to check whether program behavior satisfies specified intentions.
- We evaluate INTENTKEEPER on real-world smart contracts, demonstrating its effectiveness in validating developer intentions during the development phase.

The remainder of this paper is organized as follows. Section 2 provides background on numeric logic errors and their classification by execution semantics. Section 3 describes the design of INTENTKEEPER, including its intent model and validation engine. Section 4 presents our evaluation results. Section 5 discusses limitations and future work. Section 6 reviews related work, and Section 7 concludes.

2 Background

2.1 Numeric Logical Error Classification by Execution Semantics in Smart Contracts

2.2 Difficulties in resolving logic errors and their root causes

2.3 Proposed Solution

3 The Design of IntentKeeper

3.1 System Architecture

3.2 Intent Model

3.3 Validation Engine

4 Evaluation

5 Discussion

6 Related Works

6.1 Logic Error Detection in Solidity

6.2 Specification-based Smart Contract Verification

6.3 Tools for strengthening security during the development phase

6.4 Code-comment inconsistency in solidity

7 Conclusion

References

- Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3623302>
- Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). <https://doi.org/10.1109/TSE.2025.3555622>
- Jeon, I., et al.: SolQDebug: Debug Solidity Quickly for Interactive Immediacy in Smart Contract Development (2025). <https://doi.org/10.21203/rs.3.rs-8077153/v1>
- Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. Empirical Software Engineering 29(1), 13 (2024). <https://doi.org/10.1007/s10664-023-10398-5>

Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. *IEEE Transactions on Software Engineering* 50(6), 1360–1373 (2024). <https://doi.org/10.1109/TSE.2024.3383422>